

ID	Positive Feedback	Constructive Feedback	Grade
214005209	Good OOP & Polymorphism: Solid Game_Piece hierarchy encoding instant kill vs. multi-hit. Placement retry loop handles overcrowded boards cleanly. Commands (show/quit) and win conditions verified working.	Spec Deviations & OOP Flaws: Missing required Enum usage and start() method. General is hardcoded as a string rather than a subclass, breaking polymorphism. Missing test coverage for game logic/CLI.	77
213022726	Exceptional Architecture: Excellent use of __init_subclass__ registry allowing new pieces with zero core edits. Clean geometric rotation (_rotate90), Enums, custom exceptions, and type hints. Full E2E CLI test coverage.	Coverage & Minor Spec Gaps: Interactive game loop (_take_turn) lacks automated test coverage. Jet piece allows 4 rotations vs. 2 in README.	97
207621558	Highly Extensible Design: Clean PieceType dataclass architecture with zero piece-specific branching. Matches requested API (start()). Robust placement bounds checks under stress test.	Missing Test File: Submitted as single file with zero automated unit tests. Does not track already-fired cells (allows duplicate shots). Misleading win message when General dies.	86
212455497	Spec-Exact & Well-Tested: Matches requested public API. Excellent use of frozen dataclasses, Enums, and clean placement logic. 29 test cases covering flow, placement, and win states.	Minor Spec/Doc Gaps: Jet piece implements taller 6-cell cross instead of 5-cell plus. Hardcoded 3-layer assumption. Missing docstrings on several public methods.	94
207231523	Solid Core Logic: Clean bounds/occupancy validation before placement. All piece shapes correct. Thorough NumPy-style docstrings. CLI commands don't derail turn order.	Fake Enums & Hardcoded Core: Uses plain strings mimicking Enums (Signal.KILL). Hardcodes parameters per piece in constructor, making expansion tedious. Zero test coverage for CLI/inputs.	80
323832329	Working Core Mechanics: Piece shapes, orientations, and win conditions verified working end-to-end. CLI commands work mid-turn.	Procedural Design & File Issues: Submitted as non-.py file without proper OOP/Enums. Coordinate parsing breaks on 2-digit coordinates (10,1,1). Missing docstrings and turn/hit tests.	70
302233341	Rock-Solid Core Architecture: Complete separation of validation and mutation. Clean Enum usage. Uniquely verified extensibility via core-code grep test. Mid-turn commands handled cleanly.	Arbitrary Constraints: Enforces undocumented minimum 4x4 board size limit. show command hides user's own pieces without explanation. CLI entry point lacks test coverage.	96
207185851	Masterclass Extensibility & Testing: __init_subclass__ auto-registers pieces. Excellent use of IntEnum and NumPy shape rotation. Injected input_fn/print_fn allows full CLI testability.	UI/UX Flaws: Command validation checks player O's board regardless of whose turn it is. Unclear coordinate input requirements. No visual distinction in show between hit/unhit cells.	94
208448811	Clean Modular Architecture: Extensible via single PieceSpec registry call. Auto-derives orientations from base shapes. Full IntEnum/Enum/dataclass usage with type hints and doctests.	Rigid CLI & Non-Standard Shape: Jet shape does not match README ASCII art. Coordinate parser format is overly rigid. Unit tests lack fixtures/parameterization, causing boilerplate.	93
305249641	Strong Math & Extensibility: Auto-derives orientations via np.rot90 deduplication. Consolidated _validate_cells check handles bounds, layers, and overlaps cleanly. Good test coverage for mechanics.	Deviations & Unfinished Code: Jet shape deviates from spec. Re-firing an already-hit cell gives no feedback. Commented out signal message dict leaves raw Signal.HIT printed in UI.	92